# Python Exercises (RL-Module2): Tabular SARSA, Tabular Q-learning, and DQN

DRL Part - Module 2 (Exercises)

March 4, 2026

**How to use this sheet.** Each exercise is designed to be coded in Python and then analyzed. You are given: (i) a goal, (ii) what to implement, (iii) what to plot/report, and (iv) code-level hints + commented starter code. **Your task is to complete the TODO parts, run experiments, and explain results.**

**Recommended setup (minimal).**

- Python 3.10+
- `pip install gymnasium torch numpy matplotlib`
- `pip install gymnasium[classic-control]` (CartPole)
- `pip install gymnasium[toy-text]` (CliffWalking)

**Gymnasium interface you will use (read this once).**

- Create an environment: `env = gym.make("CliffWalking-v0")` or `gym.make("CartPole-v1")`
- Reset: `obs, info = env.reset(seed=0)`.
- Step: `obs2, reward, terminated, truncated, info = env.step(action)`.
- A full episode ends when `done = terminated or truncated`.
  - `terminated`: the environment's natural terminal condition.
  - `truncated`: time limit or external truncation (still treat as terminal for bootstrapping).
- Inspect spaces:
  - Discrete states/actions: `env.observation_space.n`, `env.action_space.n`.
  - Continuous state vectors: `env.observation_space.shape[0]`.
- Observation type differs by environment:
  - **Toy-text (CliffWalking):** `obs` is an *integer state index*.
  - **Classic-control (CartPole):** `obs` is a *float vector* (NumPy array) you feed to a neural network.

**Quick "does the env run?" test (recommended before coding the algorithms).**

```
import gymnasium as gym
env = gym.make("CliffWalking-v0")
obs, info = env.reset(seed=0)
for t in range(5):
   a = env.action_space.sample()
   obs, r, terminated, truncated, info = env.step(a)
   done = terminated or truncated
   print(t, obs, r, done)
   if done:
```

```
      obs, info = env.reset()
env.close()
```

**Submission format (suggested).**

- **Code:** one notebook per exercise, or three `.py` scripts.
- **Plots:** saved as `.png` and included in your report.
- **Short report:** 0.5–1 page per exercise (key plots + interpretation).

**What you must complete (important).** The code shown in this sheet is *starter code* and hints. To complete the exercises you must:

- Fill in all `TODO` sections (do not leave them commented out).
- Run the required experiments (baseline + comparisons), ideally with multiple random seeds.
- Save and include the required plots.
- Write short answers interpreting what you observe (connect back to on-policy/off-policy and stabilization in DQN).

---

# Exercise 1 (Tabular): SARSA vs Q-learning on CliffWalking

## Learning goals

- Implement **tabular** action-value learning from scratch: $Q(s, a)$ as a table.
- Understand **on-policy** (SARSA) vs **off-policy** (Q-learning) through the *different targets*.
- Practice $\epsilon$-greedy exploration and interpret learning curves and learned policies.

## Key conceptual difference (in one sentence)

- **SARSA (on-policy):** backup uses the *next action you actually take* under $\epsilon$-greedy.
- **Q-learning (off-policy):** backup uses the *greedy best next action* via $\max_{a'} Q(s', a')$, even if behavior is exploratory.

## Environment (CliffWalking-v0)

`CliffWalking-v0` is a small **discrete** gridworld (`gymnasium[toy-text]`), so $Q(s, a)$ can be stored as a table. Important details you should know before coding:

- **State:** an integer index $s \in \{0, \ldots, nS - 1\}$. The grid is typically $4 \times 12$ (so $nS = 48$).
- **Actions:** 4 discrete moves. In Gym toy-text, the common mapping is:

$$0 = \text{UP}, \quad 1 = \text{RIGHT}, \quad 2 = \text{DOWN}, \quad 3 = \text{LEFT}.$$

- **Rewards:** usually $-1$ per step; stepping into the "cliff" gives a large negative reward (often $-100$) and resets you.
- **Episode ends:** when you reach the goal or hit a time-limit truncation. Treat `terminated OR truncated` as terminal for bootstrapping.

**Useful helper: decode state index to grid cell.**

```
# For a 4x12 grid (common CliffWalking layout)
def state_to_rc(s, n_cols=12):
   r = s // n_cols
   c = s % n_cols
   return r, c
```

**Before training:** run 1–2 random episodes and print transitions so you trust what the environment returns.

### Update rules (what you are coding)

- **SARSA:**
$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big(r + \gamma Q(s',a') - Q(s,a)\Big).$$
- **Q-learning:**
$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big(r + \gamma \max_{a'} Q(s',a') - Q(s,a)\Big).$$

### What you must implement

1. A $Q$-table $Q[s,a]$ initialized to zeros.
2. An $\epsilon$-greedy policy for action selection (behavior policy).
3. Two trainers: `train_sarsa(...)` and `train_qlearning(...)`.
4. Logging: episode return; count of "cliff falls" (heuristic based on reward).
5. Plot and compare SARSA vs Q-learning (same hyperparameters).

### Functions you are expected to write/use (Exercise 1)

When you complete the TODOs, you should end up with these functions (names can vary, but keep the *roles*):

- `eps_greedy(Q, s, eps, rng) -> int`: returns an action index using $\epsilon$-greedy with random tie-breaking.
- `train_sarsa(...)`: runs episodes, updates $Q$ with the SARSA target $r + \gamma Q(s', a')$, logs returns.
- `train_qlearning(...)`: runs episodes, updates $Q$ with the Q-learning target $r + \gamma \max_{a'} Q(s', a')$, logs returns.
- `eval_tabular_greedy(env_id, Q, ...)` (used in Exercise 2): runs episodes with $\epsilon = 0$ and **no learning**.

**Tip:** start by making `eps_greedy` correct and deterministic (seeded), then plug it into both trainers.

### Task checklist (do these in order)

1. **Create two files:** `ex1_sarsa.py` and `ex1_qlearning.py` (or one notebook with two sections).
2. **Implement** `eps_greedy(Q,s,eps)` with random tie-breaking for arg max.
3. **Implement** `train_sarsa(...)` exactly with the SARSA target $r + \gamma Q(s', a')$ where $a'$ is chosen by the same $\epsilon$-greedy policy.
4. **Implement** `train_qlearning(...)` exactly with the Q-learning target $r+\gamma \max_{a'} Q(s', a')$.
5. **Run a baseline:** $\alpha = 0.1$, $\gamma = 0.99$, $\epsilon$ linearly decays from 1.0 to 0.05 over 3000 episodes; train for 10,000 episodes.
6. **Plot** episode return (raw + moving average) for both methods on the same figure.
7. **Plot** cliff falls per 100 episodes (use your reward-based heuristic).

8. **Optional but recommended:** print the greedy policy arrows (from the final $Q$-table) to visually compare the learned routes.
9. **Write 5–10 lines:** explain the difference in risk-taking using the on-policy vs off-policy targets.

### What to submit

- Code (notebook or `.py`).
- Two plots: (i) episode return vs episode, (ii) cliff falls per 100 episodes.
- 5–10 lines: which policy is safer/riskier and why (connect to on-/off-policy).

### Practical hints (common pitfalls)

- **Tie-breaking for** $\arg\max$**:** randomly choose among equal best actions; otherwise you can get strange artifacts.
- **Terminal handling:** when the episode ends, the bootstrap term should be zero (target is just $r$).
- **If curves are flat:** lower $\alpha$ (e.g., 0.05), increase episodes, or use $\epsilon$-decay.
- **Sanity check:** print the first few TD errors $\delta$ to confirm updates change $Q$.

### Commented starter code (complete the TODOs)

```
import numpy as np
import gymnasium as gym
import matplotlib.pyplot as plt

# ----------------------
# Helper: moving average for smoother curves
# ----------------------
def moving_avg(x, w=200):
    x = np.asarray(x, dtype=np.float32)
    if len(x) < w:
        return x
    return np.convolve(x, np.ones(w)/w, mode="valid")

# ----------------------
# Helper: epsilon schedule (optional but recommended)
# Linear decay: eps_start -> eps_end over decay_episodes
# ----------------------
def eps_by_episode(ep, eps_start=1.0, eps_end=0.05, decay_episodes=3000):
    frac = min(1.0, ep / decay_episodes)
    return eps_start + frac * (eps_end - eps_start)

# ----------------------
# Epsilon-greedy action selection with random tie-break
# ----------------------
def eps_greedy(Q, s, eps, rng):
    # Q: (n_states, n_actions) table
    # s: integer state index
    if rng.random() < eps:
        # Explore: random action
        return int(rng.integers(Q.shape[1]))

    # Exploit: greedy action with tie-breaking
    q_row = Q[s]
    max_q = q_row.max()
```

```
    best_actions = np.flatnonzero(q_row == max_q)
    return int(rng.choice(best_actions))

# ----------------------
# SARSA trainer (on-policy)
# ----------------------
def train_sarsa(env_id="CliffWalking-v0", episodes=10000,
                alpha=0.1, gamma=0.99,
                eps_start=1.0, eps_end=0.05, decay_episodes=3000,
                seed=0):
    env = gym.make(env_id)
    rng = np.random.default_rng(seed)

    # Tabular Q(s,a) initialization
    nS = env.observation_space.n
    nA = env.action_space.n
    Q = np.zeros((nS, nA), dtype=np.float32)

    returns, cliff_falls = [], []

    for ep in range(episodes):
        eps = eps_by_episode(ep, eps_start, eps_end, decay_episodes)

        s, _ = env.reset(seed=seed + ep)
        a = eps_greedy(Q, s, eps, rng) # action chosen by behavior policy

        done = False
        ep_ret = 0.0
        ep_cliff = 0

        while not done:
            s2, r, terminated, truncated, _ = env.step(a)
            done = terminated or truncated
            ep_ret += r

            # CliffWalking: falling off often yields a large negative reward (e.g.,
                -100)
            if r <= -100:
                ep_cliff += 1

            if done:
                # Terminal transition: no bootstrap term
                td_target = r
                td_error = td_target - Q[s, a]
                Q[s, a] += alpha * td_error
                break

            # SARSA is on-policy: choose next action a2 using SAME eps-greedy behavior
                policy
            a2 = eps_greedy(Q, s2, eps, rng)

            # Target uses Q(s2, a2) (NOT max over actions)
            td_target = r + gamma * Q[s2, a2]
            td_error = td_target - Q[s, a]
            Q[s, a] += alpha * td_error

            # Move forward in time: (s,a) <- (s2,a2)
            s, a = s2, a2

        returns.append(ep_ret)
        cliff_falls.append(ep_cliff)

    env.close()
```

```
    return Q, np.array(returns), np.array(cliff_falls)

# ----------------------
# Q-learning trainer (off-policy)
# ----------------------
def train_qlearning(env_id="CliffWalking-v0", episodes=10000,
                    alpha=0.1, gamma=0.99,
                    eps_start=1.0, eps_end=0.05, decay_episodes=3000,
                    seed=0):
    env = gym.make(env_id)
    rng = np.random.default_rng(seed)

    nS = env.observation_space.n
    nA = env.action_space.n
    Q = np.zeros((nS, nA), dtype=np.float32)

    returns, cliff_falls = [], []

    for ep in range(episodes):
        eps = eps_by_episode(ep, eps_start, eps_end, decay_episodes)

        s, _ = env.reset(seed=seed + ep)
        done = False
        ep_ret = 0.0
        ep_cliff = 0

        while not done:
            # Behavior: still eps-greedy (we explore while learning)
            a = eps_greedy(Q, s, eps, rng)

            s2, r, terminated, truncated, _ = env.step(a)
            done = terminated or truncated
            ep_ret += r

            if r <= -100:
                ep_cliff += 1

            if done:
                td_target = r
            else:
                # Off-policy target: greedy max over next actions
                td_target = r + gamma * np.max(Q[s2])

            td_error = td_target - Q[s, a]
            Q[s, a] += alpha * td_error

            s = s2

        returns.append(ep_ret)
        cliff_falls.append(ep_cliff)

    env.close()
    return Q, np.array(returns), np.array(cliff_falls)

# ----------------------
# TODO: Run both methods with same hyperparams and plot
# ----------------------
# Qs, ret_s, cliff_s = train_sarsa()
# Qq, ret_q, cliff_q = train_qlearning()
# plt.figure()
# plt.plot(ret_s, alpha=0.2); plt.plot(ret_q, alpha=0.2)
# plt.plot(np.arange(199, len(ret_s)), moving_avg(ret_s, 200), label="SARSA MA200")
```

```
# plt.plot(np.arange(199, len(ret_q)), moving_avg(ret_q, 200), label="Q-learning
    MA200")
# plt.legend(); plt.grid(True); plt.xlabel("Episode"); plt.ylabel("Return")
# plt.show()
```

### Suggested experiments

- Compare SARSA vs Q-learning under the same $\alpha, \gamma$ and $\epsilon$-schedule.
- Try $\epsilon \in \{0.05, 0.1, 0.3\}$ (or different decay speeds) and report how risk changes.
- Run 3 random seeds and report mean and variability (learning curves can be noisy).

---

## Exercise 2 (Evaluation): Make on-policy vs off-policy visible

### Learning goals

- Separate **behavior policy** (data generation) from **evaluation policy**.
- Plot **training return** (with exploration) vs **evaluation return** (greedy, $\epsilon = 0$).
- Explain why SARSA may look different under greedy evaluation if training keeps a fixed $\epsilon$.

### What you must do

1. Train SARSA and Q-learning using a behavior exploration rate (e.g., $\epsilon_{\text{train}} = 0.2$ or a decay schedule).
2. Every $K$ episodes (e.g., 200), evaluate the current greedy policy with $\epsilon_{\text{eval}} = 0$ for 20 episodes and record mean return.
3. Plot for each method: training return vs episode, and greedy evaluation return vs episode.
4. Explain (5–10 lines): why can training return be worse than greedy evaluation? which method improves faster in greedy evaluation and why?

### Functions you are expected to write/use (Exercise 2)

- `eval_tabular_greedy(env_id, Q, episodes, seed)`: runs the environment with $\epsilon = 0$ and returns the mean (and optionally std) return.
- Small modifications to `train_sarsa` and `train_qlearning`: add a block that calls `eval_tabular_greedy` every $K$ episodes and stores the results in arrays for plotting.

**Minimum requirement:** your plots must clearly distinguish *training return* (exploration on) and *evaluation return* (greedy, no learning).

### Task checklist (do these in order)

1. Reuse your Exercise 1 trainers, but add two logging arrays: `train_returns` (every episode) and `eval_returns` (every $K$ episodes).
2. Choose $K = 200$ and evaluate for $M = 20$ greedy episodes with $\epsilon_{\text{eval}} = 0$.
3. For fairness, use the *same* evaluation seeds for SARSA and Q-learning (e.g., seed 999 + episode index).

4. Plot **two curves per method**: (i) training return, (ii) evaluation return (mean over $M$ episodes).
5. Answer: (a) Why does training return look noisier/worse? (b) Why can SARSA's greedy evaluation lag if $\epsilon$ is kept fixed during training?

### Important evaluation rules

- During evaluation: **no learning** (do not update $Q$).
- During evaluation: **greedy actions only** ($\epsilon = 0$).
- Use a fixed seed for evaluation episodes to reduce randomness when comparing methods.

### Commented evaluation helper code

```python
def eval_tabular_greedy(env_id, Q, episodes=20, seed=123):
    # Evaluate a tabular policy greedily (epsilon = 0).
    # Returns: (mean_return, std_return)
    import gymnasium as gym
    env = gym.make(env_id)

    rets = []
    rng = np.random.default_rng(seed) # for deterministic tie-breaking

    for i in range(episodes):
        s, _ = env.reset(seed=seed + i)
        done = False
        ep_ret = 0.0

        while not done:
            # Greedy action + tie-break
            q = Q[s]
            max_q = q.max()
            best = np.flatnonzero(q == max_q)
            a = int(rng.choice(best))

            s, r, terminated, truncated, _ = env.step(a)
            done = terminated or truncated
            ep_ret += r

        rets.append(ep_ret)

    env.close()
    return float(np.mean(rets)), float(np.std(rets))

# How to call during training:
# if (ep + 1) % K == 0:
# mean_ret, std_ret = eval_tabular_greedy("CliffWalking-v0", Q, episodes=20, seed
    =999)
# eval_curve.append(mean_ret)
# eval_x.append(ep + 1)
```

---

## Exercise 3 (Deep RL): Implement DQN + run ablations

### Learning goals

- Implement a minimal but correct DQN: online net, target net, replay buffer, $\epsilon$-greedy.
- Understand why DQN needs stabilizers (target network + replay buffer).

- Use Huber loss and observe differences vs MSE under ablations.
- Compare **Adam (adaptive)** vs **plain SGD (non-adaptive)** and explain the effect on learning speed/stability.

### Environment options (and what you need to know)

- **Recommended (fast, no extra deps):** `CartPole-v1` (`gymnasium[classic-control]`).
  - **Observation:** a 4D float vector (NumPy array) $[\, x, \dot{x}, \theta, \dot{\theta} \,]$.
  - **Actions:** 2 discrete actions (0 = push left, 1 = push right).
  - **Reward:** typically +1 per time step (so higher return means balancing longer).
  - **Episode end:** when the pole falls too much, the cart goes out of bounds, or a time limit truncates the episode (often 500 steps).
- **Optional (harder):** `LunarLander-v2`. This may require extra dependencies (Box2D).
  - Install attempt: `pip install gymnasium[box2d]` (may require system packages depending on OS).
  - Use it only after your CartPole DQN baseline is working.

**Environment inspection snippet (run once).**

```
import gymnasium as gym
env = gym.make("CartPole-v1")
obs, _ = env.reset(seed=0)
print("obs shape:", obs.shape) # should be (4,)
print("n_actions:", env.action_space.n) # should be 2
env.close()
```

### Functions and environment calls you will use (Exercise 3)

A correct DQN implementation is mostly about wiring a few functions together with the Gymnasium API:

- **Environment calls:**
  - `obs, info = env.reset(seed=...)`
  - `obs2, r, terminated, truncated, info = env.step(a)`
  - `done = terminated or truncated` (use this for terminal masking)
- **Functions/classes you must complete:**
  - `ReplayBuffer.push(...)` and `ReplayBuffer.sample(batch_size)` (random minibatches).
  - `QNet.forward(x)` returning $Q(s, \cdot) \in \mathbb{R}^{n\_actions}$.
  - `select_action(qnet, state, eps, ...)` implementing $\epsilon$-greedy.
  - `dqn_train_step(...)`: builds the target $y$, computes loss (Huber/MSE), does backprop + optimizer step.
  - `eval_dqn(...)`: runs greedy episodes ($\epsilon = 0$) with **no learning**.
  - `run_dqn(...)`: the main loop that (i) collects transitions, (ii) stores them in replay, (iii) trains periodically, (iv) updates the target network, and (v) logs returns.

### Step-by-step implementation plan (strongly recommended)

Follow these milestones to avoid "silent bugs":

1. **Milestone A (env + shapes):** run 5 random steps and print `obs.shape` and `n_actions`.
2. **Milestone B (network):** create `QNet` and verify `qnet(torch.randn(1, obs_dim)).shape == (1, n_actions)`.
3. **Milestone C (replay):** push 200 random transitions, sample a batch, and print shapes (expect $B \times obs\_dim$ for states).
4. **Milestone D (one train step):** call `dqn_train_step` once and ensure it returns a finite loss.
5. **Milestone E (baseline learning):** run baseline DQN on CartPole; evaluation return should increase over time.
6. **Milestone F (ablation):** change exactly one component (target/replay/loss/optimizer) and re-run with the same budget.

### DQN target (with terminal masking)

For a transition $(s, a, r, s', done)$,

$$y = r + (1 - \texttt{done})\,\gamma \max_{a'} Q_{\theta^-}(s', a').$$

Then update $Q_\theta$ by minimizing a loss on the executed action:

$$\mathcal{L}(\theta) = \ell\Big(y,\ Q_\theta(s, a)\Big).$$

### Huber loss definition (and why it helps)

Let the prediction error be $e = y - \hat{y}$ (in DQN: $e = y - Q_\theta(s, a)$). For a threshold $\delta > 0$, the Huber loss is:

$$L_\delta(e) = \begin{cases} \frac{1}{2}e^2, & \text{if } |e| \le \delta, \\ \delta\left(|e| - \frac{1}{2}\delta\right), & \text{if } |e| > \delta. \end{cases}$$

**Interpretation:** quadratic (MSE-like) for small errors, linear (MAE-like) for large errors. This makes learning more robust to occasional large TD errors.

### Optimizer ablation: Adam vs plain SGD (what to expect)

- **Adam** adapts the step size per-parameter using running averages of gradients and squared gradients. This often helps in deep RL where gradients are noisy.
- **SGD** uses a fixed global step size (optionally with momentum). With the same learning rate as Adam it can be much slower; with a larger learning rate it can become unstable.
- For a fair comparison, keep everything identical *except* the optimizer, and try a small learning-rate sweep for SGD (e.g., $10^{-2}, 5 \cdot 10^{-3}, 10^{-3}$).

### What is an ablation study? (meaning and why we do it)

An **ablation study** is a controlled experiment where you *remove or replace one component at a time* while keeping everything else fixed. The goal is to identify which components are truly responsible for performance or stability. In deep RL, this is essential because many design choices (replay buffer, target network, optimizer, loss) interact.

- **Baseline:** choose a "reasonable" default DQN configuration that learns (e.g., replay + target + Huber + Adam).
- **One change at a time:** create an "ablated" version where *only one* component is changed (e.g., remove target net).
- **Same protocol:** same environment steps, same evaluation frequency, same random seeds (or multiple seeds).
- **Compare metrics:** evaluation return vs steps, stability (variance across seeds), and time-to-threshold (e.g., steps to reach a return of 400 on CartPole).

### Recommended ablation protocol (for this exercise)

1. First get the **baseline** to learn on `CartPole-v1` (target+replay+Huber+Adam).
2. Then run ablations where you change **exactly one** of: target network, replay buffer, loss, optimizer.
3. Run at least 3 random seeds and report mean/variation (deep RL is noisy).

### Ablation experiments (do at least 3)

- **No target network:** use online net for the target too (often unstable).
- **No replay:** train on the most recent transition only (high correlation, less stable).
- **Huber vs MSE** + optional gradient clipping.
- **Optimizer (Adam vs SGD):** run the same DQN with `optim.Adam` and `optim.SGD` ("without Adam"). Keep everything else fixed; expect to retune the learning rate for SGD.
- **Target copy period** $N \in \{100, 1000, 10000\}$.
- **Exploration schedule:** decay too fast vs too slow.

### Task checklist (do these in order)

1. **Implement the baseline DQN:** replay buffer ON, target network ON, Huber loss ON, optimizer = Adam.
2. **Complete the TODO logs** in `run_dqn`: `episode_returns`, `eval_returns`, `eval_steps`, and the evaluation block every 5000 steps.
3. **Plot baseline learning:** greedy evaluation return vs environment steps. (A healthy baseline on CartPole should typically approach 400–500 return.)
4. **Run optimizer ablation:** switch `optimizer="sgd"` ("without Adam") and sweep learning rate $\in \{10^{-2}, 5 \cdot 10^{-3}, 10^{-3}\}$ (optionally momentum 0.9). Keep everything else fixed.
5. **Run two more ablations:** remove the target network and remove replay (or swap Huber to MSE).
6. **Report:** one figure that overlays *all* conditions (baseline + ablations) with the same axes.
7. **Explain:** which component mattered most for stability, and why.

### Suggested experiment matrix (baseline + ablations)

Keep **all** hyperparameters and the training budget fixed, and only change the indicated component. A good default is: `total_steps=200000`, `target_update=1000`, evaluate every 5000 steps (10 episodes, $\epsilon = 0$).

| Condition | Replay buffer | Target net | Loss | Optimizer |
|---|---|---|---|---|
| Baseline | ON | ON | Huber | Adam |
| A1: No target net | ON | OFF | Huber | Adam |
| A2: No replay | OFF | ON | Huber | Adam |
| A3: "Without Adam" | ON | ON | Huber | SGD |
| A4: MSE loss | ON | ON | MSE | Adam |

Run at least three seeds (e.g., 0, 1, 2) and compare (i) final evaluation return, (ii) learning speed, and (iii) stability/variance.

### Debug checklist (if "it doesn't learn")

- Are you masking terminals correctly? If `done=1`, then $y = r$.
- Are your tensor shapes correct? `qnet(s)` is $B \times A$; `gather` gives $B \times 1$.
- Do you warm up replay before training (e.g., first 1000 steps)?
- Are you evaluating with $\epsilon = 0$ and **not** updating the network during evaluation?

### Commented minimal DQN skeleton (complete the TODOs)

```python
import random
from collections import deque
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import gymnasium as gym

# ----------------------
# Replay Buffer: stores transitions and returns random minibatches
# ----------------------
class ReplayBuffer:
    def __init__(self, capacity=100_000):
        self.buf = deque(maxlen=capacity)

    def push(self, s, a, r, s2, done):
        self.buf.append((s, a, r, s2, done))

    def sample(self, batch_size):
        batch = random.sample(self.buf, batch_size)
        s, a, r, s2, done = map(np.array, zip(*batch))
        return s, a, r, s2, done

    def __len__(self):
        return len(self.buf)

# ----------------------
# Q Network (MLP)
# Input: state vector
# Output: Q-values for all discrete actions
# ----------------------
class QNet(nn.Module):
```

```
    def __init__(self, obs_dim, n_actions, hidden=128):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(obs_dim, hidden), nn.ReLU(),
            nn.Linear(hidden, hidden), nn.ReLU(),
            nn.Linear(hidden, n_actions)
        )

    def forward(self, x):
        return self.net(x)

@torch.no_grad()
def select_action(qnet, state, eps, n_actions, device):
    # epsilon-greedy behavior
    if random.random() < eps:
        return random.randrange(n_actions)
    s = torch.tensor(state, dtype=torch.float32, device=device).unsqueeze(0)
    q = qnet(s)
    return int(torch.argmax(q, dim=1).item())

def dqn_train_step(qnet, target_qnet, opt, batch, gamma, device, use_huber=True):
    s, a, r, s2, done = batch

    # Tensors with correct shapes
    s = torch.tensor(s, dtype=torch.float32, device=device)
    a = torch.tensor(a, dtype=torch.int64, device=device).unsqueeze(1)
    r = torch.tensor(r, dtype=torch.float32, device=device).unsqueeze(1)
    s2 = torch.tensor(s2, dtype=torch.float32, device=device)
    done = torch.tensor(done, dtype=torch.float32, device=device).unsqueeze(1)

    # Q(s,a) from online net via gather -> (B,1)
    q_sa = qnet(s).gather(1, a)

    # Target y uses target net + terminal masking
    with torch.no_grad():
        max_next_q = target_qnet(s2).max(dim=1, keepdim=True)[0]
        y = r + (1.0 - done) * gamma * max_next_q

    # Huber loss (SmoothL1) vs MSE
    if use_huber:
        loss = nn.SmoothL1Loss(beta=1.0)(q_sa, y)
    else:
        loss = nn.MSELoss()(q_sa, y)

    opt.zero_grad(set_to_none=True)
    loss.backward()
    nn.utils.clip_grad_norm_(qnet.parameters(), 10.0)
    opt.step()
    return float(loss.item())

@torch.no_grad()
def eval_dqn(env_id, qnet, episodes=10, seed=0, device="cpu"):
    env = gym.make(env_id)
    rets = []
    for i in range(episodes):
        s, _ = env.reset(seed=seed + i)
        done = False
        ep_ret = 0.0
        while not done:
            st = torch.tensor(s, dtype=torch.float32, device=device).unsqueeze(0)
            a = int(torch.argmax(qnet(st), dim=1).item())
            s, r, terminated, truncated, _ = env.step(a)
            done = terminated or truncated
```

```
            ep_ret += r
        rets.append(ep_ret)
    env.close()
    return float(np.mean(rets)), float(np.std(rets))

def run_dqn(env_id="CartPole-v1", total_steps=200_000, batch_size=64,
            gamma=0.99, lr=1e-3, buffer_size=100_000,
            learning_starts=1000, train_freq=1, target_update=1000,
            eps_start=1.0, eps_end=0.05, eps_decay_steps=50_000,
            seed=0, use_target=True, use_replay=True, use_huber=True,
            optimizer="adam", sgd_momentum=0.0):

    env = gym.make(env_id)
    obs_dim = env.observation_space.shape[0]
    n_actions = env.action_space.n

    random.seed(seed); np.random.seed(seed); torch.manual_seed(seed)
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    qnet = QNet(obs_dim, n_actions).to(device)
    target_qnet = QNet(obs_dim, n_actions).to(device)
    target_qnet.load_state_dict(qnet.state_dict())

    # ----------------------
    # Optimizer ablation: Adam vs SGD ("without Adam")
    # NOTE: plain SGD often needs a larger lr than Adam.
    # Try: Adam lr=1e-3; SGD lr in {1e-2, 5e-3, 1e-3} and (optionally) momentum=0.9.
    # ----------------------
    if str(optimizer).lower() == "adam":
        opt = optim.Adam(qnet.parameters(), lr=lr)
    elif str(optimizer).lower() == "sgd":
        opt = optim.SGD(qnet.parameters(), lr=lr, momentum=sgd_momentum)
    else:
        raise ValueError("optimizer must be 'adam' or 'sgd'")
    rb = ReplayBuffer(buffer_size)

    s, _ = env.reset(seed=seed)
    ep_ret = 0.0

    # TODO: create logs: episode_returns, eval_returns, eval_steps
    # episode_returns = []
    # eval_returns = []
    # eval_steps = []

    for t in range(1, total_steps + 1):
        # Linear epsilon decay in steps
        frac = min(1.0, t / eps_decay_steps)
        eps = eps_start + frac * (eps_end - eps_start)

        a = select_action(qnet, s, eps, n_actions, device)
        s2, r, terminated, truncated, _ = env.step(a)
        done = terminated or truncated
        ep_ret += r

        if use_replay:
            rb.push(s, a, r, s2, float(done))

        # Training
        if t > learning_starts and t % train_freq == 0:
            if use_replay and len(rb) >= batch_size:
                batch = rb.sample(batch_size)
            else:
                # No replay ablation: repeat latest transition (intentionally poor)
```

```
            batch = (np.repeat(s[None,:], batch_size, axis=0),
                   np.repeat(np.array([a]), batch_size),
                   np.repeat(np.array([r]), batch_size),
                   np.repeat(s2[None,:], batch_size, axis=0),
                   np.repeat(np.array([float(done)]), batch_size))

            tgt = target_qnet if use_target else qnet
            loss = dqn_train_step(qnet, tgt, opt, batch, gamma, device, use_huber=
                use_huber)

        # Target update
        if use_target and (t % target_update == 0):
            target_qnet.load_state_dict(qnet.state_dict())

        # Episode reset
        if done:
            # TODO: episode_returns.append(ep_ret)
            s, _ = env.reset(seed=seed + t)
            ep_ret = 0.0
        else:
            s = s2

        # TODO: Evaluate every N steps (e.g., 5000)
        # if t % 5000 == 0:
        # mean_eval, _ = eval_dqn(env_id, qnet, episodes=10, seed=999, device=device)
        # eval_returns.append(mean_eval); eval_steps.append(t)

    env.close()
    return qnet
```

### What to submit

- Code + plots: evaluation return vs environment steps (overlay different ablations).
- Include at least one plot comparing **Adam vs SGD** ("without Adam") under the same settings (report the learning rates you used).
- Short report: what you changed, what happened, and why you think it happened.

**Bonus (optional):** Implement Double DQN (DDQN) by selecting the next action with the online net but evaluating it with the target net.